NepalCRM

Role, Permission & Module System — Complete Guide for Frontend

1. The Foundation: Permissions

A **permission** is the smallest unit of access control. Every action a user can take in the system is backed by exactly one permission.

Each permission has these fields:

- `code` — unique identifier, e.g. `CASE_MANAGEMENT:VIEW`
- `action` — the operation: `VIEW`, `CREATE`, `EDIT`, `DELETE`, `ACCESS`, etc.
- `scope` — controls **which role types** can receive this permission (see below)

Permission Scopes

Scope	Who can have it	Example
GLOBAL	SUPER_ADMIN only	System configuration, audit settings
TENANT	FIRM_ADMIN and below	Most module permissions (CASE_MANAGEMENT, EMPLOYEE, BILLING, etc.)
ASSIGNED	ADVOCATE / PARALEGAL and below	View assigned cases, upload documents
OWN	CLIENT only	View own client records

> **Important:** A role can only receive permissions at or below its ceiling. FIRM_ADMIN ceiling is `TENANT` — so it can receive TENANT, ASSIGNED, or OWN permissions, but NOT GLOBAL. Trying will return a **403** with a clear message.

Example Permissions in the System

```
CASE_MANAGEMENT:ACCESS    (scope: TENANT)
CASE_MANAGEMENT:VIEW      (scope: TENANT)
CASE_MANAGEMENT:CREATE    (scope: TENANT)
```

```
CASE_MANAGEMENT:EDIT      (scope: TENANT)
CASE_MANAGEMENT:DELETE    (scope: TENANT)
EMPLOYEE:ACCESS           (scope: TENANT)
EMPLOYEE:VIEW             (scope: TENANT)
EMPLOYEE:CREATE           (scope: TENANT)
USER_MANAGEMENT:VIEW      (scope: TENANT)
BILLING:ACCESS            (scope: TENANT)
BILLING:VIEW              (scope: TENANT)
DOCUMENT_MANAGEMENT:ACCESS (scope: TENANT)
CALENDAR:ACCESS            (scope: TENANT)
REPORTS:VIEW               (scope: TENANT)
AUDIT:VIEW                 (scope: TENANT)
```

2. The Next Layer: Roles

A **role** is a named collection of permissions. When a user is assigned a role, they inherit all the permissions in that role.

System Roles (Pre-seeded on Startup)

Role Code	For	Included Module Permissions	Scope Ceiling
`SUPER_ADMIN`	Platform administrators	Everything — all modules, all actions	GLOBAL
`FIRM_ADMIN`	Firm administrators	Case Mgmt, Employee, Client, Billing, Documents, Calendar, Reports, Audit	TENANT
`ADVOCATE`	Lawyers in a firm	Case view/edit, document upload, calendar	ASSIGNED
`PARALEGAL`	Support staff	Case view, document upload, calendar view	ASSIGNED
`CLIENT`	Clients of the firm	Own case view, own document view	OWN

How Role Cloning Works (When a Firm is Created)

1. Super Admin creates a new firm via `POST /api/v1/firms`
2. The backend **clones** each system role template for that specific firm
3. Each cloned role stores `parentRoleId` pointing back to the original system template
4. The cloned role keeps the same permissions as the template — but can be modified later
5. The cloned role is assigned to users within that firm

> **Key rule:** A cloned role cannot receive permissions that exceed its parent's **scope ceiling**. FIRM_ADMIN ceiling is TENANT, so it cannot receive GLOBAL-scoped permissions. This is enforced at the API level — a 403 is returned if violated.

3. The Top Layer: Modules + Authorization

A **module** is a logical grouping in the UI — Case Management, Employee Management, Billing, Calendar, etc.

When the frontend loads the application, it calls:

```
GET /api/v1/me
```

The response includes the user's profile and their entire permission set:

```
{
  "username": "admin@firm.com",
  "userType": "FIRM_USER",
  "roleCode": "FIRM_ADMIN",
  "permissions": [
    "CASE_MANAGEMENT:ACCESS",
    "CASE_MANAGEMENT:VIEW",
    "CASE_MANAGEMENT:CREATE",
    "CASE_MANAGEMENT:EDIT",
    "CASE_MANAGEMENT:DELETE",
    "EMPLOYEE:ACCESS",
    "EMPLOYEE:VIEW",
    "EMPLOYEE:CREATE",
    "BILLING:ACCESS",
    ...
  ]
}
```

Your Frontend Job

Read the `permissions` array from `/me` and use it to decide what to show or hide in the UI.

```
// Example: Sidebar rendering logic
const permissions = meResponse.data.permissions;

// Show "Case Management" in sidebar only if user has ACCESS
if (permissions.includes("CASE_MANAGEMENT:ACCESS")) {
  renderSidebarItem("Case Management");
}
```

```
// Inside Case Management page, show "Create Case" button only if user has CREATE
if (permissions.includes("CASE_MANAGEMENT:CREATE")) {
  renderCreateButton();
}

// Hide "Employee Management" entirely if user has no ACCESS
if (!permissions.includes("EMPLOYEE:ACCESS")) {
  hideSection("Employee Management");
}
```

> **Double enforcement:** The backend also checks the same permission on every API call. Showing a button the user cannot use is a bad UX, but even if someone calls the API directly, the backend will reject the request. There is no way to bypass.

4. Complete Flow: From Permission Creation to User Seeing It

Step 1 — Super Admin creates a permission:

```
POST /api/v1/admin/permissions
{
  "code": "CASE_MANAGEMENT:ACCESS",
  "action": "ACCESS",
  "scope": "TENANT"
}
```

Step 2 — Super Admin assigns it to system role templates:

```
POST /api/v1/admin/roles/permissions
{
  "roleId": "<SUPER_ADMIN-template-id>",
  "permissionIds": ["<new-permission-id>"]
}

POST /api/v1/admin/roles/permissions
{
  "roleId": "<FIRM_ADMIN-template-id>",
  "permissionIds": ["<new-permission-id>"]
}
```

Since it has `scope: TENANT`, the ceiling allows it on FIRM_ADMIN. If it had `scope: GLOBAL`, only SUPER_ADMIN could receive it.

Step 3 — The firm admin refreshes their firm-scoped role:

```
PUT /api/v1/admin/roles
{
  "id": "<firm-scoped-role-id>",
  "name": "FIRM_ADMIN",
  "code": "FIRM_ADMIN",
  "permissionIds": [
    "...all 50 permission IDs including the new one..."
  ]
}
```

This single call updates the role name AND replaces all its permissions at once.

Step 4 — Users are automatically updated:
The backend increments an internal `permissionVersion` for every user who holds that role. Their cached permissions are invalidated. They do NOT need to log out and back in.

Step 5 — Frontend reads `/me` and the module appears:
The user's `/me` response now includes `CASE_MANAGEMENT:ACCESS`. Your frontend checks the permissions array, finds the ACCESS permission, and shows the Case Management module in the sidebar. Done.

5. The Scope Ceiling Fix (What Changed)

Before

The ceiling check compared **permission IDs** against the parent role's exact set in the database. If a permission was created after the system roles were seeded, it was NOT in the parent's set. So even a legitimate TENANT-scoped permission by a FIRM_ADMIN role would get **403 blocked** unless the admin also remembered to update the system template.

After

The ceiling check compares **permission scope levels** against the role type's ceiling. Any TENANT-scoped permission is automatically assignable to any FIRM_ADMIN role — no need to update system templates.

Error Response When Ceiling is Violated

```
{
  "success": false,
  "message": "Permission 'USER_MANAGEMENT' (scope: GLOBAL) exceeds
              your role's ceiling. This scope is not available
              for role type 'FIRM_ADMIN'.",
  "responseCode": 403
}
```

Solution: Either change the permission's scope to `TENANT` (via `PUT /api/v1/admin/permissions`), or accept that it belongs to a higher role tier.

6. API Reference for Your Role/Permission UI

Method	Endpoint	Purpose
`GET`	`/api/v1/admin/permissions`	List all permissions (for dropdowns / permission picker)
`POST`	`/api/v1/admin/permissions`	Create a new permission
`PUT`	`/api/v1/admin/permissions`	Update a permission (change name, scope, etc.)
`GET`	`/api/v1/admin/roles`	List all roles with their permissions
`POST`	`/api/v1/admin/roles`	Create or update a role (includes `permissionIds`)
`DELETE`	`/api/v1/admin/roles/{id}`	Delete a non-system role
`GET`	`/api/v1/admin/roles/{id}/permissions`	Get permissions for a specific role
`POST`	`/api/v1/admin/roles/permissions`	Assign permissions to a role (dedicated endpoint)
`GET`	`/api/v1/me`	Get current user's profile + role + permissions array

`GET`	`/api/v1/firm/roles`	List firm-scoped roles (for firm admin UI)
`PUT`	`/api/v1/firm/roles/{id}/permissions`	Update a firm role's permissions (firm admin)
`GET`	`/api/v1/firm/roles/{id}/available-permissions`	Get available permissions within ceiling (for permission picker dropdown)

Frontend UI Checklist

- Fetch all permissions → show in a searchable, grouped grid/module list
- Let super admin pick permissions for a role → send as `permissionIds` array
- Fetch role list → display with permission count badge
- Read `/me` on login → store permissions array in app state → gate every UI element
- Use a `hasPermission(code)` helper function everywhere in the frontend

7. Example: Building a Permission Picker UI

Your permission assignment screen should:

1. **Fetch all permissions** from `GET /api/v1/admin/permissions`
2. **Group them by module** (the part before the colon in the code, e.g. `CASE_MANAGEMENT`, `EMPLOYEE`, `BILLING`)
3. **Fetch the role's current permissions** from `GET /api/v1/admin/roles/{id}/permissions`
4. **Show checkboxes** grouped by module, with current permissions pre-checked
5. **On save**, send the complete list of checked permission IDs via `POST /api/v1/admin/roles`

Example grouped data structure:

```
[
  {
    "module": "CASE_MANAGEMENT",
    "permissions": [
      { "id": "...", "code": "CASE_MANAGEMENT:ACCESS",  "action": "ACCESS",  "checked":
      { "id": "...", "code": "CASE_MANAGEMENT:VIEW",    "action": "VIEW",    "checked":
      { "id": "...", "code": "CASE_MANAGEMENT:CREATE",  "action": "CREATE",  "checked":
      { "id": "...", "code": "CASE_MANAGEMENT:EDIT",    "action": "EDIT",    "checked":
      { "id": "...", "code": "CASE_MANAGEMENT:DELETE",  "action": "DELETE",  "checked":
```

```
    ]
  },
  {
    "module": "EMPLOYEE",
    "permissions": [
      { "id": "…", "code": "EMPLOYEE:ACCESS", "action": "ACCESS", "checked": true },
      { "id": "…", "code": "EMPLOYEE:VIEW",   "action": "VIEW",   "checked": true },
       …
    ]
  }
]
```